

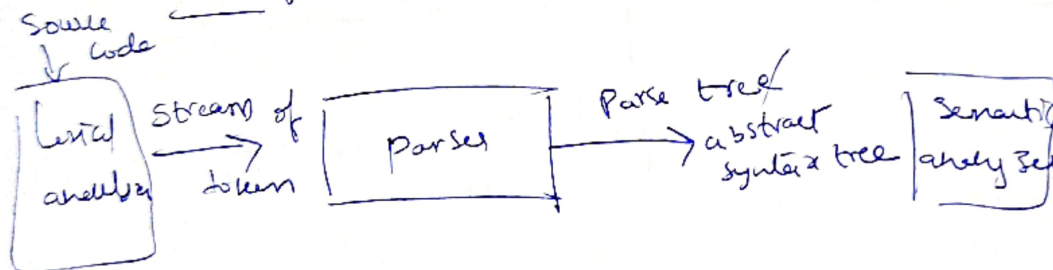
Syntax Analysis

Syntax analysis is 2nd phase of Compiler that checks whether sequence of tokens generated by lexical analyzer follows the grammatical rules of programming language

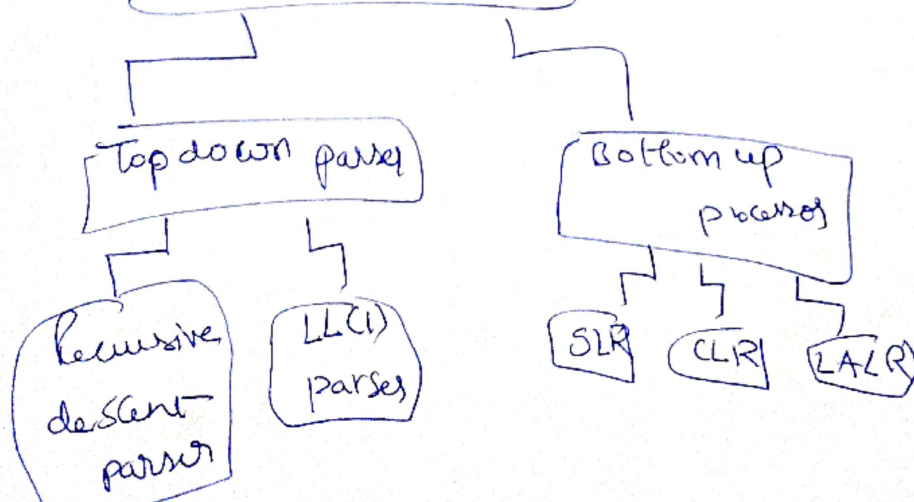
ex: `int a=10;`
follows grammar

`int = a 10;`
doesn't follow grammar

Role of Syntax Analyzer



Types of parsers



Context free grammar (CFG)

Introduction

To check syntax, Compiler needs
Set of grammatical rules.

These rules are called Grammar

@ CFG: Collection of production
rules used to describe syntax of
a programming language

→ each production rules replaces a
single non-terminal with combination of
terminals and/or non terminals.

$$G = (V, T, P, S)$$

↑
Terminal

↓
non terminal

↓
production rule

Start symbol

Ex:-

$$\begin{aligned} &E \rightarrow E + T \\ &E \rightarrow T \\ &T \rightarrow id \end{aligned}$$

production rules

E, T - non terminals

$id, +$ - terminals

Start symbol - E

derivation:-

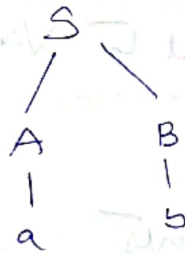
process of generating string using
production rules.

ex:-

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$



final string: ab

parse tree:-

parser constructs tree called

parse tree.

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow id$

string id + id



Backus-Naur Form (BNF)

BNF is formal mathematical notation used to describe syntax or grammar of programming language.

→ acts as precise blueprint that parser tools read to validate code correctness

Core Syntax Components

< > :- enclose non-terminal symbols.

::= (definition operator)
means "is defined as" or
"can be replaced with"

| (vertical bar) :- represents "OR"
separating alternative choices
for expansion

plain text / quotes :- represents
terminal symbols
ex: +, if, 5 etc..

Examples

1. Defining postal digit and number

$$\langle \text{digit} \rangle ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"$$
$$\langle \text{number} \rangle ::= \langle \text{digit} \rangle | \langle \text{digit} \rangle \langle \text{number} \rangle$$
$$\langle \text{list} \rangle \rightarrow \langle \text{item} \rangle \langle \text{item} \rangle \langle \text{list} \rangle$$

Extended Backus - Normal form (EBNF)

EBNF is extension of BNF that provides concise and powerful grammar notation.

[] - optional

{ } - Repeating (one or more times)

() - grouping choice

| - choice

Ex: 1) $\langle \text{digit} \rangle ::= [0-9]$

2) $\langle \text{list} \rangle ::= \text{item} \{ , \text{item} \}$

Print (a,b,c) →

3) $\langle \text{function} \rangle ::=$

$\langle \text{identifier} \rangle$

$(\langle \text{expr} \rangle \{ , \text{expr} \})$

Left recursion

A grammar is left recursive if non-terminal symbol A can eventually derive string that starts with A itself

$$A \rightarrow A\alpha/\beta$$

This creates infinite recursive loop without ever consuming an input token

Eliminating left recursion

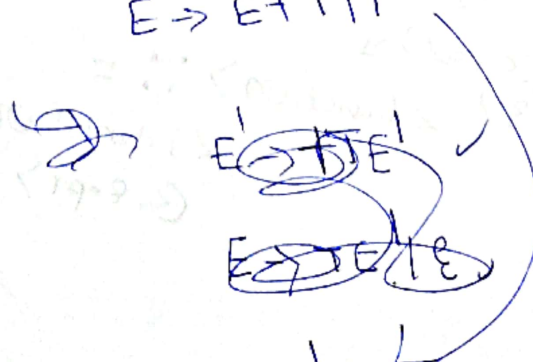
$$\text{if } A \rightarrow A\alpha/\beta$$

then convert to

$$\begin{aligned} A' &\rightarrow \alpha A' / \epsilon \\ A &\rightarrow \beta A' \end{aligned}$$

ex

$$E \rightarrow E + T / T$$



$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

Left factoring

left factoring is required when two or more production options for a non-terminal share Common leftmost prefix

formula : $A \rightarrow \alpha\beta_1 | \alpha\beta_2$

if it sees prefix α , it cannot decide which rule to use

Transformation rule

$A \rightarrow \alpha\beta_1 | \alpha\beta_2$

$A \rightarrow \alpha A'$

$\emptyset$

$A' \rightarrow \beta_1 | \beta_2$

Ex:-

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow A$

$S \rightarrow \text{else } S | \epsilon$

$S \rightarrow \text{if } E \text{ then } S S' | \epsilon$

$S \rightarrow A$

Top down parsing

parsing technique that constructs a parse tree from root node down to the leaf nodes (input tokens)

Recursive descent parser

RDP is Top-down parsing technique in which set of recursive procedures is used to parse the input string

- One function is written for each non-terminal in the grammar
- parse starts from start symbol and recursively calls functions corresponding to grammar rules
- It constructs parse tree from root to leaves

Back tracking :- It is trial and error technique used in top down parsing

- if attempted production rule fails to match input string, parser undoes the step and reverts to previous decision point to try alternative rules until successful parse tree is built.

Ex¹
= given grammar

$$E \rightarrow E+T \mid T$$

$$\sigma \quad T \rightarrow id$$

1) after preprocessing

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow id$$

work flow

Input

↓
Call ~~E~~ E()

↓
Call T()

↓
match id

↓
Call E'()

↓
if '+' found

↓
Call T()

↓
repeat

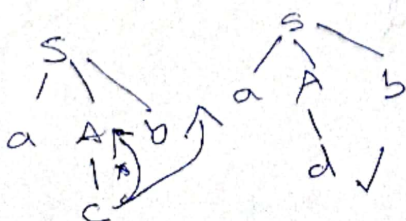
↓
Accept

Back tracking Example

$$S \rightarrow aAB$$

$$A \rightarrow c \mid d$$

input - acdb



```

E( )
{
    TC();
    E'( );
}

E'( )
{
    if (input == '+' )
    {
        match ('+');
        TC();
        E'( );
    }
}

TC( )
{
    match @ (id);
}

```



if input id id

step by step parsing:-

Call E()



Call TC()

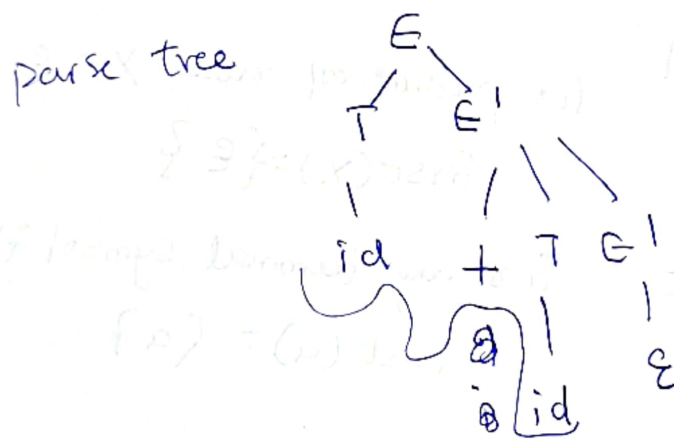


match first id



Call E'()

↓
 match +
 ↓
 Call T()
 ↓
 match second id
 ↓
 Call E'()
 ↓
 ε
 ↓
 accepted



match() → compares current input token
 with expected token.

If it matches, advances to next input
 symbol, otherwise reports an error

First function

first(α) is set of terminal symbols that begin in strings derived from α

Ex:- for $A \rightarrow abc \mid def \mid ghi$,
 $\text{First}(A) = \{a, d, g\}$

Rules for calculating first function

Rule-1 for production rule $X \rightarrow \epsilon$
 $\text{first}(X) = \{\epsilon\}$

Rule-2 for any terminal symbol a
 $\text{first}(a) = \{a\}$

Rule-3 for $X \rightarrow Y_1 Y_2 Y_3$
for $\text{first}(X)$
• if $\epsilon \notin \text{first}(Y_1)$, then $\text{first}(X)$
 $= \text{first}(Y_1)$

• if $\epsilon \in \text{first}(Y_1)$, then
 $\text{first}(X) = \{\text{first}(Y_1) - \epsilon\}$
for $\text{first}(Y_2 Y_3) \cup \text{first}(Y_2 Y_3)$

if $\epsilon \notin \text{first}(Y_2)$, then $\text{first}(Y_2 Y_3)$
 $= \text{first}(Y_2)$

if $\epsilon \in \text{first}(Y_2)$,
 $\text{first}(Y_2 Y_3) = \{\text{first}(Y_2) - \epsilon\}$
 $\cup \text{first}(Y_3)$

Ex-1

$$\begin{aligned} S &\rightarrow aBD \\ B &\rightarrow aD \\ D &\rightarrow b \end{aligned}$$

$$\underline{\text{first}(S) = a}$$

$$\text{first}(B) = \text{first}(D) \\ = b$$

$$\text{first}(D) = b$$

Ex-2

$$A \rightarrow bdc | \epsilon$$

$$B \rightarrow \cancel{ad} AD | b$$

$$D \rightarrow d$$

$$\text{first}(A) = \{b, \epsilon\}$$

$$\text{first}(B) = \text{first}(AD)$$

$$= \{\text{first } A - \epsilon\} \cup \text{first}(D)$$

$$= \{b, d\}$$

$$\text{first}(D) = d$$

Ex-3

$$A \rightarrow BCD | a | \epsilon$$

$$B \rightarrow ACd$$

$$C \rightarrow \epsilon | c$$

$$\text{first} = \{a, \epsilon\}$$

(A)

$$\text{first}(B) = \text{first}(ACd)$$

$$= a \cup \text{first}(C) = \{a, c\} \cup$$



Follow function

$\text{follow}(a)$ is set of terminal symbols that appear immediately to right of a

Rule 1 :- for start symbol S
place $\$$ in $\text{follow}(S)$

Rule 2 :- for any production rule
 $A \rightarrow aB$

$$\text{follow}(B) = \text{follow}(A)$$

Rule 3 :- for any production rule
 $A \rightarrow aB\beta$

• if $\epsilon \notin \text{first}(\beta)$, then $\text{follow}(B)$
 $= \text{first}(\beta)$

• if $\epsilon \in \text{first}(\beta)$, then $\text{follow}(B)$
 $= \{ \text{first}(\beta) - \epsilon \}$
 $\cup \text{follow}(A)$

Problem 1

$$S \rightarrow aBDh$$

$$B \rightarrow dc$$

$$C \rightarrow dc/\epsilon$$

$$D \rightarrow EF$$

$$E \rightarrow g/\epsilon$$

$$F \rightarrow f/\epsilon$$

first function

$$\text{first}(S) = \{a\}$$

$$\text{first}(B) = \{d\}$$

$$\text{first}(C) = \{d, \epsilon\}$$

$$\text{first}(D) = \text{first}(E)$$

$$= \{g\} \cup \text{first}(F)$$

$$= \{g\} \cup \{f, \epsilon\}$$

$$= \{g, f, \epsilon\}$$

$$\text{first}(E) = \{g, \epsilon\}$$

$$\text{first}(F) = \{f, \epsilon\}$$

~~problem 2~~

Q

first(CDh)
 $\{s, f, h\}$

$$\text{follow}(S) = \{ \$ \}$$

$$\begin{aligned} \text{follow}(B) &= \text{first}(CDh) \\ &= \{s, f, h\} \cup \text{follow}(CDh) \\ &= \{s, f, h\} \end{aligned}$$

$$= \text{first}(CDh) = \{s, f, h\}$$

$$\text{follow}(C) = \text{follow}(B)$$

$$\text{follow}(D) = \{h\}$$

$$\text{follow}(E) = \text{first}(F)$$

$$= \{t\} \cup \text{follow}(D)$$

$$= \{t, h\}$$

$$\begin{aligned} \text{follow}(F) &= \text{follow}(D) \\ &= \{h\} \end{aligned}$$

Ex 2

$$S \rightarrow (L) | a$$

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' | \epsilon$$

$$\text{first}(S) = \{ (, a \}$$

$$\text{first}(L) = \text{first}(SL')$$

$$= \{ (, a \}$$

$$\text{first}(L') = \{ , , \epsilon \}$$

$$\text{follow}_{(S)} = \{ \$ \} \cup \text{first}(L') - \epsilon \} \cup \text{follow}(L)$$

$$= \{ \$ \} \cup \{ , \} \cup \text{follow}(L)$$

$$= \{ \$, , \} \cup \text{follow}(L')$$

$$= \{ \$, , \}$$

$$\text{follow}(L) = \{ , \}$$

$$\text{follow}(L') = \text{follow}(L)$$

$$= \{ , \}$$

Q.3

$$S \rightarrow AaAb \mid BbBa$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

$$\text{first}(S)$$

$$= \{\text{first}(A) - \epsilon\} \cup \text{first}(a) \\ \cup \{\text{first}(B) - \epsilon\} \cup \text{first}(b)$$

$$\text{first}(A) = \{\epsilon\}$$

$$\text{first}(B) = \{\epsilon\}$$

$$\text{first}(S) = \{a, b\}$$

$$\text{follow}(S) = \{\$ \}$$

$$\text{follow}(A) = \{a, b\}$$

$$\text{follow}(B) = \{a, b\}$$

Ex 4

$$\begin{aligned} E &\rightarrow E+T / T \\ T &\rightarrow T * F / F \\ F &\rightarrow (E) / id \end{aligned}$$

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' / \epsilon \end{aligned}$$

$$\begin{aligned} T &\rightarrow FT' \\ T' &\rightarrow *FT' / \epsilon \\ F &\rightarrow (E) / id \end{aligned}$$

$$\begin{aligned} \text{first}(E) &= \text{first}(T) \\ &= \text{first}(F) = \{ (, id \} \end{aligned}$$

$$\text{first}(F) = \{ (, id \}$$

$$\text{first}(T) = \{ (, id \}$$

$$\text{first}(E') = \{ +, \epsilon \}$$

$$\text{first}(T') = \{ *, \epsilon \}$$

$$\text{follow}(E) = \{ \$,) \}$$

$$\begin{aligned} \text{follow}(F) &= \{ \text{first}(T') - \epsilon \} \cup \text{follow}(T) \\ &= \{ *, +, \$,) \} \end{aligned}$$

$$\begin{aligned} \text{follow}(T) &= \{ \text{first}(E') - \epsilon \} \\ &\cup \text{follow}(E) \end{aligned}$$

$$= \{ +, \$,) \}$$

$$\begin{aligned} \text{follow}(E') &= \text{follow}(E) = \{ \$,) \} \end{aligned}$$

Q.5

$$S \rightarrow ACB \mid c b B \mid B a$$

$$A \rightarrow d a \mid B c$$

$$B \rightarrow g \mid \epsilon$$

$$C \rightarrow h \mid \epsilon$$

$$\text{first}(S) =$$

$$\text{first}(C) = \{h, \epsilon\}$$

$$\text{first}(B) = \{g, \epsilon\}$$

$$\text{first}(A) = \{d\} \cup \text{first}(B) \cup \text{first}(C)$$

$$= \{d, g, h, \epsilon\}$$

$$\text{first}(S) = \{\text{first}(A) - \epsilon\}$$

$$\cup \{\text{first}(C) - \epsilon\}$$

$$\cup \text{first}(B)$$

$$\cup \{\text{first}(C) - \epsilon\}$$

$$\cup \text{first}(B)$$

$$\cup \{\text{first}(B) - \epsilon\} \cup \text{first}(a)$$

$$= \{d, g, h\} \cup \{b\} \cup \{h, b\}$$

$$\cup \{g\}$$

$$= \{d, g, h, b\}$$

$$\text{follow}(S) = \{ \$ \}$$

$$\text{follow}(A) = \text{first}(CB) \\ = \{ \text{first}(C) - \epsilon \} \cup \{ \text{first}(B) - \epsilon \} \cup \text{follow}(S)$$

$$= \{ h, a, \epsilon \} \cup \{ \} \\ = \{ h, a, \epsilon \}$$

$$\text{follow}(B) = \text{follow}(S) \\ \cup \text{first}(CA) \\ \cup \{ \text{first}(C) - \epsilon \} \\ \cup \text{follow}(A)$$

$$= \{ \epsilon, a, h, g, \epsilon \}$$

$$\text{follow}(C) = \{ \text{first}(B) - \epsilon \} \cup \text{follow}(S) \\ \cup \text{first}(CA) \cup \text{follow}(A)$$

$$= \{ g, \epsilon, b, h \}$$

LL(1) parser

LL(1) parser is top-down parser
that reads input from β left to right
produces leftmost derivation
and uses 1 look ahead symbol to
choose correct production

Steps for constructing LL(1)

Parsing table

step 1:- find first sets

step 2:- find follow sets

step 3:- for each production

$A \rightarrow \alpha$ place it in $M[A, a]$

for every $a \in \text{first}(\alpha)$

step 4:- if ϵ belongs to first

place production in $M[A, b]$

or for $b \in \text{follow}(A)$

Parsing algorithm

step 1: push $\$$ and start symbol in the stack $\$S$

step 2: read first input symbol

step 3: $\rightarrow$ if terminal match $\rightarrow$ pop stack and move input pointer

$\rightarrow$ if non terminal - use parsing table

step 4: continue until

stack = $\$$

input = $\$$

then Accept, otherwise reject

Ex 1

$S \rightarrow aA$

$A \rightarrow b$

input = $ab\$$

First(S) = a , First(A) = b

follow(S) = $\$$, follow(A) = $\$$

	a	b	$\$$
S	$S \rightarrow aA$		
A		$A \rightarrow b$	

<u>State</u>	<u>input</u>	<u>Action</u>
\$S	ab\$	$S \rightarrow aA$
\$A\$	ab\$	match a
\$A	b\$	$A \rightarrow b$
\$b	b\$	match b
\$	\$	Accept

ex 2

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow id$$

id id \$

$$\begin{aligned} \text{first}(E) &= \text{first}(T) = \{id\} \\ \text{first } E' &= \{+, \epsilon\} \end{aligned}$$

$$\text{follow}(E) = \{\$ \}$$

$$\begin{aligned} \text{follow}(E') &= \text{follow}(E) \\ &= \{\$ \} \end{aligned}$$

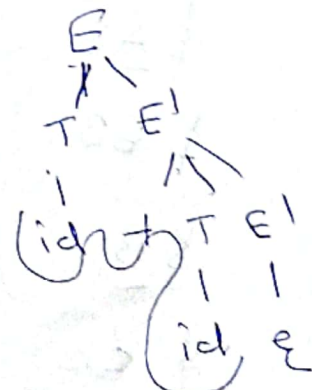
$$\text{follow}(T) = \{\text{first}(E') - \epsilon\} \cup \text{follow}(E)$$

$$\cup \{\text{first}(E') - \epsilon\} \cup \text{follow}(E')$$

$$= \{+, \$\}$$

Nonterminals	+	id	\$
E	ϵ	$E \rightarrow TE'$	
E'	$E' \rightarrow +TE'$		$E' \rightarrow \epsilon$
T		$T \rightarrow id$	

Stack	Input	Action
\$E	id+id\$	$E \rightarrow TE'$
\$TE'	id+id\$	$T \rightarrow id$
\$idE'	id+id\$	match id
\$E'	+id\$	$E' \rightarrow +TE'$
\$+TE'	+id\$	match +
\$TE'	id\$	$T \rightarrow id$
\$idE'	id\$	match id
\$E'	\$	$E' \rightarrow \epsilon$
\$	\$	accept



Ex 3:-

$S \rightarrow aAB$

$A \rightarrow b|e$

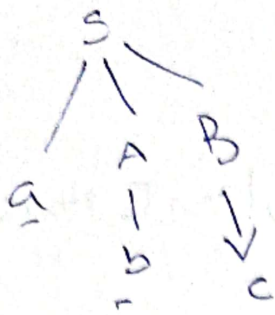
$B \rightarrow c$

input = $abc\$$

$\text{first}(S) = \{a\}$ $\text{follow}(S) = \{\$ \}$
 $\text{first}(A) = \{b, e\}$ $\text{follow}(A) = \text{first}(B) = \{c\}$
 $\text{first}(B) = \{c\}$ $\text{follow}(B) = \{\$ \}$

	a	b	c	d
S	$S \rightarrow aAB$			
A		$A \rightarrow b$ $A \rightarrow e$		
B			$B \rightarrow c$	

stack	input	action
$\$S$	$abc\$$	$S \rightarrow aAB$
$\$AB$	$bc\$$	match a
$\$AB$	$bc\$$	$A \rightarrow b$
$\$B$	$c\$$	match S
$\$B$	$c\$$	$B \rightarrow c$
$\$$	$c\$$	match C
$\$$	$\$$	accept



Bottom-up parser

starts parsing from input string
(leaves) and gradually reduces it to start symbol
(root)

Right most derivation in reverse

ex:

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow id$$

input - id + id

~~id + id~~

$$E \rightarrow E + \underline{T}$$

$$E \rightarrow E + \underline{id}$$

$$E \rightarrow \underline{T} + id$$

$$E \rightarrow \underline{id} + id$$



Shift reduce parser

Shift moves next input symbol on the stack
after shift

Stack: \$ Input: id + id \$

\$ id + id \$

Reduce replaces symbol on top of stack by corresponding non terminal

$E \rightarrow id$

Stack:
\$ id
\$ E

Reduce

Handle

substring that matches right hand side of production and can be reduced to non terminal

$E \rightarrow E + T$

$T \rightarrow id$

Stack

Stack:
 $E + (id) \rightarrow handle$

Ex:-

Parse the input idtid
for
 $E \rightarrow E+T \mid T$
 $T \rightarrow id$

Stack	input	action
\$	idtid\$	shift id
\$id	+id\$	reduce $T \rightarrow id$
\$T	+id\$	shift + reduce $E \rightarrow E+T$ shift id
\$E	id\$	
\$E+id		

Stack	input	Action
\$	idtid\$	shift id
\$id	+id\$	reduce $T \rightarrow id$
\$T	+id\$	reduce $E \rightarrow T$
\$E	+id\$	shift +
\$E+	id\$	shift id
\$E+id	\$	reduce $T \rightarrow id$
\$E+id	\$	$E \rightarrow E+T$
\$E+T	\$	Accept
\$E		

Simple LR (SLR) parser

is an efficient bottom up syntax analysis techniques used in compilers to read input streams from left-to-right and construct Rightmost derivation in reverse

→ It uses DFA built from LR(0) items but relies on Follow sets

Components of SLR Parser

- Input buffer
- Stack
- Action table
- Goto table

LR(0) item

is a production with dot(.) showing how much production has been recognized.

Ex: $E \rightarrow ET$

$\Rightarrow E \rightarrow \cdot ET$

$E \rightarrow E \cdot T$

$E \rightarrow E + \cdot T$

$E \rightarrow E + T \cdot$

Closure:-

adds all productions that may be required when dot is before a non terminal.

Grammar

$E \rightarrow \cdot T$

$T \rightarrow \cdot id$

Closure

$E \rightarrow \cdot T$

$T \rightarrow \cdot id$

Goto:-

moves the dot over grammar symbol

ex:-

$E \rightarrow \cdot T$

After reading T

$E \rightarrow T \cdot$

Construction of SLR parsing Table

Step 1:-

Augment the grammar

ex:-

$E' \rightarrow E$

Step 2:-

Construct LR(0) items

Step 3:-

find closure

Step 4:-

Construct Goto transitions

Step 5:-

Build ACTION table using follow sets

Step 6:-

Build Goto table.

Parsing Table

ACTION Table

contains

- shift
- Reduce
- Accept
- Error

GoTo Table

contains transitions for

Non terminals

Ex:



$S \rightarrow CC$

$C \rightarrow eC$

$C \rightarrow d$

$S' \rightarrow S$

Step 2: Construct LR(0) Items

$$I_0 = \text{closure}(S' \rightarrow \cdot S)$$

include all productions of S

$S \rightarrow \cdot CC$

include all productions of C

$C \rightarrow \cdot eC$

$C \rightarrow \cdot d$

I_0 $S' \rightarrow \cdot S$

$S \rightarrow \cdot CC$

$C \rightarrow \cdot eC$

$C \rightarrow \cdot d$

$$\text{Goto}(10, S) \rightarrow I_1$$

$$S' \rightarrow S$$

$$\text{Goto}(10, C) \rightarrow I_2$$

$$S \rightarrow C.C$$

$$C \rightarrow .eC$$

$$C \rightarrow .d$$

$$\text{Goto}(10, e) \rightarrow I_3$$

$$C \rightarrow e.C$$

$$C \rightarrow .ec$$

$$C \rightarrow .d$$

$$\text{Goto}(10, d) \rightarrow I_4$$

$$C \rightarrow d.$$

$$\text{Goto}(I_2, C) \rightarrow I_5$$

$$S \rightarrow CC.$$

~~$$\text{Goto}(I_2, e) \rightarrow I_6$$

$$C \rightarrow e.C$$

$$C \rightarrow .ec$$

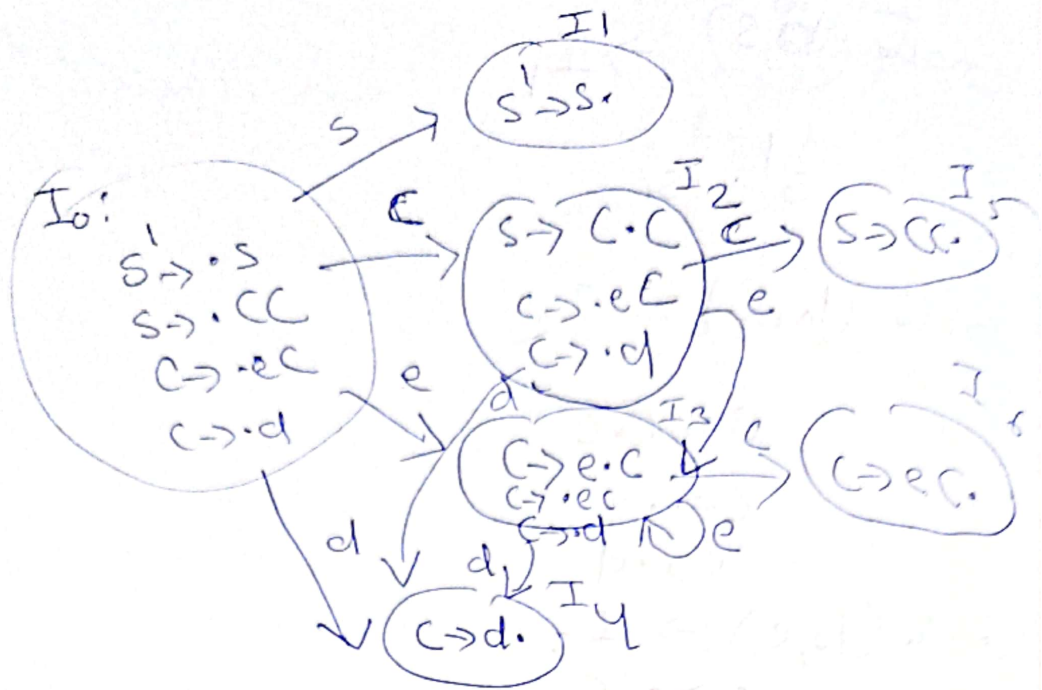
$$C \rightarrow .d$$~~

$$\text{Goto}(I_3, C) \rightarrow I_6$$

$$C \rightarrow e.C$$

~~$$\text{Goto}(I_3, e) \rightarrow I_6$$

$$C \rightarrow e.C$$~~



follow set s

$$\text{follow}(s) = \{ \$ \}$$

$$\text{follow}(c) = \text{first}(c)$$

$$= \{ c, d, \$ \}$$

~~follow set~~

Action Table

state	e	d	\$
0	s ₃	s ₄	
1	.		Accept
2	s ₃	s ₄	
3	s ₃	s ₄	
4	r(c→d)	r(c→d)	r(c→d)
5			r(s→cc)
6	r(c→ec)	r(c→ec)	r(c→ec)

Goto Table

State	S	C
0	1	2
2		5
3		6

Parsing String

Input:- dd\$

<u>Stack</u>	<u>Input</u>	<u>Action</u>
0	dd\$	shift 1
0d1	d\$	reduce R C → d
0C2	d\$	shift 4
0C2d1	d\$	reduce C → d
0C2C5	\$	reduce S → CC
0C2 0S1	\$	Accept